Cloud Build Deployment for Cloud Composer

Detailed explanation of the YAML configuration, file synchronisation, and CI/CD workflow

Purpose

This configuration deploys Airflow DAGs, reusable Python scripts, and SQL files from your Git repository into the Cloud Composer bucket. Cloud Composer then makes the DAGs available to Apache Airflow.

In one sentence

Cloud Build deploys the code; Cloud Composer and Airflow orchestrate and run the data pipeline.

1. Corrected Cloud Build configuration

Your original text contained the same configuration twice. It also joined the final destination path with a second “steps:” block. The corrected file is:

```
steps:
 - name: 'gcr.io/google.com/cloudsdktool/cloudsdk'
   entrypoint: 'bash'
   args:
    - '-c'
    - |
      gsutil -m rsync -d -r learning_airflow/dags gs://us-central1-learning-compos-6c8a22d5-bucket/dags
      gsutil -m rsync -d -r learning_airflow/scripts gs://us-central1-learning-compos-6c8a22d5-bucket/dags/scripts
      gsutil -m rsync -d -r learning_airflow/sql gs://us-central1-learning-compos-6c8a22d5-bucket/dags/sql
```

Important correction

The file must end after /dags/sql. Remove the duplicated second “steps:” block.

2. What this configuration does

Cloud Build runs one deployment step. During that step, it opens a Bash shell and executes three gsutil rsync commands:

- Synchronise the local dags folder with the Composer bucket dags folder.
- Synchronise the local scripts folder with dags/scripts in the Composer bucket.
- Synchronise the local sql folder with dags/sql in the Composer bucket.

Because Cloud Composer monitors the bucket’s dags folder, deployed Python DAG files are detected by Airflow and become visible in the Airflow user interface.

3. Line-by-line explanation

steps:

```
steps:
```

Starts the list of actions that Cloud Build must perform. This configuration contains one build step.

name:

```
name: 'gcr.io/google.com/cloudsdktool/cloudsdk'
```

Specifies the container image used for the build step. This Google Cloud SDK image includes command-line tools such as gcloud, gsutil, and bq. This deployment uses gsutil.

entrypoint:

```
entrypoint: 'bash'
```

Overrides the container’s normal starting command and opens the Bash shell. Bash lets the build step run several commands in sequence.

args:

```
args:
 - '-c'
 - |
```

The -c option tells Bash to treat the following text as shell commands. The YAML pipe symbol allows multiple commands to be written on separate lines.

4. The three synchronisation commands

Command 1: Deploy Airflow DAG files

```
gsutil -m rsync -d -r learning_airflow/dags \
  gs://us-central1-learning-compos-6c8a22d5-bucket/dags
```

Source folder: learning_airflow/dags

Destination folder: Composer bucket/dags

Example mapping:

```
learning_airflow/dags/sales_pipeline.py
-> gs://...-bucket/dags/sales_pipeline.py
```

Cloud Composer scans this destination for DAG definitions. Airflow parses the Python files and shows valid DAGs in the Airflow UI.

Command 2: Deploy reusable Python scripts

```
gsutil -m rsync -d -r learning_airflow/scripts \
  gs://us-central1-learning-compos-6c8a22d5-bucket/dags/scripts
```

Source folder: learning_airflow/scripts

Destination folder: Composer bucket/dags/scripts

Example mapping:

```
learning_airflow/scripts/load_sales.py
-> gs://...-bucket/dags/scripts/load_sales.py
```

A DAG can import a reusable module from this folder, for example:

```
from scripts.load_sales import load_sales_data
```

Command 3: Deploy SQL files

```
gsutil -m rsync -d -r learning_airflow/sql \
  gs://us-central1-learning-compos-6c8a22d5-bucket/dags/sql
```

Source folder: learning_airflow/sql

Destination folder: Composer bucket/dags/sql

Example mapping:

```
learning_airflow/sql/create_sales_table.sql
-> gs://...-bucket/dags/sql/create_sales_table.sql
```

Airflow tasks can read these files and submit the SQL to BigQuery. Keeping SQL outside the DAG file makes the pipeline easier to read and maintain.

5. Meaning of the gsutil options

Option	Meaning	Why it is used
-m	Multi-threaded transfer	Copies multiple files in parallel and can make deployment faster.
rsync	Synchronise folders	Copies new or changed files and keeps source and destination aligned.
-r	Recursive	Includes all nested files and subfolders.
-d	Delete extra destination files	Removes destination files that no longer exist in the source repository.

Be careful with -d

If a file is deleted from Git, the next build can also delete it from the Composer bucket. This keeps both locations identical, but accidental deletions can also be deployed.

6. End-to-end CI/CD workflow

The complete deployment flow is:

1	**Develop locally:** Create or update DAG, Python, and SQL files.
2	**Push to GitHub:** Commit the changes and push them to the connected repository.
3	**Trigger Cloud Build:** A repository trigger detects the push and starts the build.
4	**Run the YAML file:** Cloud Build starts the Cloud SDK container and opens Bash.
5	**Synchronise files:** gsutil copies and deletes files so the Composer bucket matches Git.
6	**Composer detects DAGs:** Cloud Composer syncs the dags bucket into the Airflow environment.
7	**Airflow orchestrates:** Airflow schedules, runs, retries, and monitors the DAG tasks.

7. Which service does what?

Service	Main responsibility	Example in this project
GitHub	Source control	Stores DAG, script, SQL, and YAML files.
Cloud Build	CI/CD deployment	Runs the YAML and copies repository files to Composer.
Cloud Storage bucket	Deployment location	Holds the DAGs, scripts, and SQL used by Composer.
Cloud Composer	Managed Airflow environment	Provides Airflow infrastructure and synchronises the DAG folder.
Apache Airflow	Orchestration	Schedules tasks, controls dependencies, retries failures, and monitors runs.
BigQuery	Data processing and storage	Executes SQL tasks launched by the DAG.

8. Simple summary

Deployment
Cloud Build copies your latest DAG, script, and SQL files from GitHub into the Cloud Composer bucket.

Orchestration
Airflow inside Cloud Composer schedules and controls the pipeline after those files have been deployed.

Key distinction
Cloud Build does not schedule the DAG. It deploys the DAG. Airflow schedules and runs it.

9. Practical checks after deployment

- Confirm the Cloud Build run completed successfully.
- Open the Composer bucket and verify that the files appear under dags, dags/scripts, and dags/sql.
- Open the Airflow UI and confirm the DAG is visible.
- Check for DAG import errors if it does not appear.
- Trigger the DAG manually once before relying only on its schedule.